

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 1 — LANGCHAIN FUNDAMENTALS

Prerequisites: Phase 0 finished — both `test_gemini.py` and `test_ollama.py` ran successfully.

Targets: LangChain 1.x · langchain-core · Runnable interface · LCEL · the `|` operator

Estimated time for this phase: 60–90 minutes

Where this phase sits

You've set up your environment. Now you'll learn the one idea everything else is built on: the **Runnable**, and the **LCEL** language for wiring Runnables together with the  pipe.

Ph 0 ✓ **Environment Setup** — complete

Ph 1 **LangChain Fundamentals** ← you are here

What LangChain is/isn't, the Runnable interface, LCEL and the  pipe operator, your first chain.

Ph 2 **Working with Models** — invoke / stream / batch, temperature & parameters.

Ph 3 **Prompts & Output Parsers** — templates, message roles, few-shot, structured output.

Ph 4 **Chains & LCEL Composition** — parallel, routing, custom functions, debugging.

Ph 5–8 **Memory · Documents · Embeddings · RAG**

Ph 9–12 **Tools & Agents · LangGraph · Deployment · Capstone**

Phase 1 · LangChain Fundamentals

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Explain in plain words what LangChain is and isn't; describe the **Runnable** contract that every piece shares; read and write an **LCEL** chain using the `|` operator; and build your first real chain — `prompt | model | parser` — in both Gemini and Ollama, swapping one for the other by changing a single line.

In this phase

- 1.1 — What LangChain *is* (and what it is *not*)
- 1.2 — The Runnable: one interface to rule them all
- 1.3 — LCEL and the `|` pipe operator
- 1.4 — Your first chain: `prompt | model | parser` (Gemini)
- 1.5 — The same chain, locally (Ollama) — the payoff
- 1.6 — A shortcut: `init_chat_model`
- 1.7 — Common errors & troubleshooting
- 1.8 — Mini practice exercise
- Terms introduced this phase

1.1 — What LangChain *is* (and what it is *not*)

In Phase 0 you called two different models and noticed the code was nearly identical. That is not a coincidence — it's the whole reason LangChain exists.

THE ONE-SENTENCE DEFINITION

LangChain is a framework that gives every model, prompt, parser, data source, and tool the same plug shape — so you can snap them together into a pipeline and swap any part without rewriting the rest.

What it *is*:

- A **common interface** over many providers (Gemini, OpenAI, Anthropic, local Ollama, ...). Learn it once, use it everywhere.
- A set of **composable building blocks** — prompt templates, output parsers, retrievers, tools — that all fit together.
- A **"glue" layer** for the boring-but-necessary plumbing: formatting prompts, parsing replies, retrying, streaming, batching, tracing.

What it is *not*:

- **Not a model.** LangChain doesn't contain an AI. It calls models that live elsewhere (Google's servers, or your machine via Ollama).
- **Not magic / not required.** You can call any model with plain HTTP. LangChain just removes repetitive code and makes parts interchangeable.
- **Not only for chatbots.** It's used for extraction, classification, search, RAG, agents, and data pipelines.

TERM: FRAMEWORK

A **framework** is a reusable skeleton of code you build *inside*. It provides the structure and common parts; you fill in the specifics. LangChain provides the "plug shape" and plumbing; you provide the prompts, data, and logic.

1.2 — The Runnable: one interface to rule them all

The "common plug shape" has a name: the **Runnable**. Almost every object in LangChain — a model, a prompt template, an output parser, a retriever, a whole chain — is a Runnable. Being a Runnable means it promises to support the same handful of methods:

Method	What it does	Input → Output
<code>.invoke(x)</code>	Run once on a single input, wait for the full result.	one in → one out
<code>.stream(x)</code>	Run once, but yield the result in pieces as it's produced (great for typing-effect UIs).	one in → chunks
<code>.batch([x1, x2, ...])</code>	Run on many inputs efficiently (in parallel under the hood).	many in → many out
<code>.ainvoke</code> / <code>.astream</code> / <code>.abatch</code>	The same three, but async (for high-concurrency apps). Same idea — ignore until you need them.	—

WHY THIS MATTERS SO MUCH

Because *everything* is a Runnable, everything speaks the same three verbs. Once you know `.invoke()`, you can invoke a model, a prompt, a parser, or an entire 10-step pipeline — identically. And because they share an interface, they **connect to each other**. That connection is the next section.

Here's the interface with *no AI at all* — just to see the contract. A `RunnableLambda` wraps any plain Python function so it becomes a Runnable:

```
from langchain_core.runnables import RunnableLambda

add_one = RunnableLambda(lambda x: x + 1)  # wrap a function as a Runnable
mul_two = RunnableLambda(lambda x: x * 2)
```

```
print(add_one.invoke(1))      # 2    -> one input, one output
print(add_one.batch([1, 2, 3])) # [2, 3, 4] -> many at once
```

Both functions are now `Runnables`, so both support `.invoke` and `.batch` for free. Next we connect them.

1.3 — LCEL and the `|` pipe operator

TERM: LCEL

LCEL = LangChain Expression Language. It's not a new programming language — it's just the *style* of connecting `Runnables` together using the `|` (pipe) symbol. `a | b` means "run `a`, then feed its output into `b`."

If you've used a Unix terminal, this is the same idea as `cat file | grep error | wc -l` — each stage's output flows into the next. In LangChain:

```
input → | add_one | → output → | mul_two | → final
      |           |
      +-----+
      chain = add_one | mul_two
```

```
chain = add_one | mul_two      # pipe two Runnables into one new Runnable

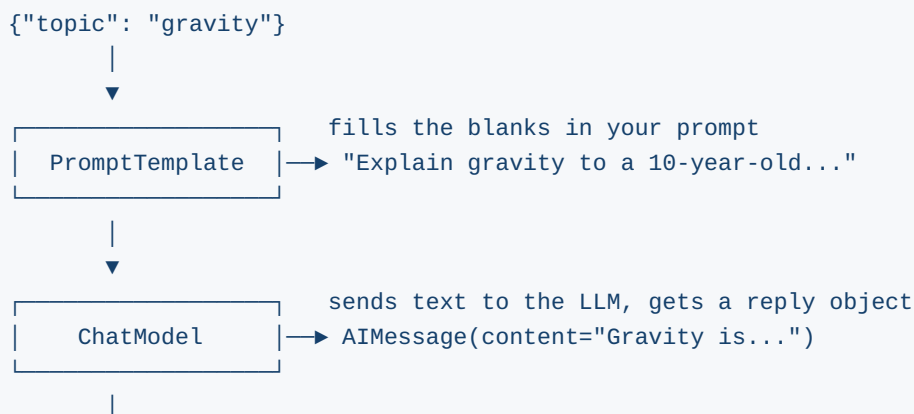
print(chain.invoke(3))        # (3 + 1) * 2 = 8
print(chain.batch([1, 2, 3])) # [4, 6, 8]
```

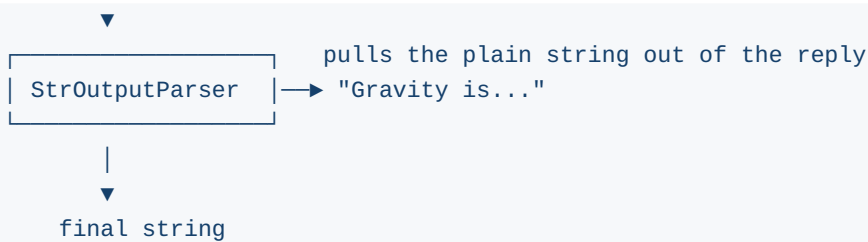
THE KEY INSIGHT

`add_one | mul_two` is **itself a Runnable**. A chain of `Runnables` is just a bigger `Runnable`. That's why you can `.invoke()` or `.batch()` the whole pipeline exactly like a single piece — and why you can pipe chains into other chains. This is the engine of the entire framework.

1.4 — Your first chain: `prompt | model | parser` (Gemini)

Now the real thing. A typical LangChain chain has three `Runnables` piped together:





Create `chain_gemini.py`:

```

from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv() # 1

prompt = ChatPromptTemplate.from_template( # 2
    "Explain {topic} to a 10-year-old in 2 short sentences."
)
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.3) # 3
parser = StrOutputParser() # 4

chain = prompt | model | parser # 5

result = chain.invoke({"topic": "gravity"}) # 6
print(result) # 7

```

Line-by-line

Line	Explanation
1	Loads your <code>GOOGLE_API_KEY</code> from <code>.env</code> (see Phase 0). Must run before creating the model.
2	<code>ChatPromptTemplate.from_template(...)</code> makes a reusable prompt with a blank, <code>{topic}</code> . It's a Runnable: input a dict, output a formatted prompt. (Full detail in Phase 3.)
3	Builds the Gemini model Runnable. <code>temperature=0.3</code> keeps answers fairly focused (Phase 2 covers this dial).
4	<code>StrOutputParser()</code> is a Runnable that takes the model's message object and returns just its <code>.content</code> string — so you don't have to write <code>.content</code> yourself.
5	The chain. Pipe the three Runnables together. Output type of each matches the input type of the next: dict → prompt → message → string.
6	<code>.invoke({"topic": "gravity"})</code> runs the <i>whole</i> pipeline. The dict fills <code>{topic}</code> ; the result flows all the way through.
7	Because the parser ran last, <code>result</code> is already a clean string — no <code>.content</code> needed.

Run it:

```
python3 chain_gemini.py
```

Expected output (wording will vary):

```
Gravity is an invisible pull that the Earth uses to keep everything close to
the ground, like a giant magnet for objects. It's the reason a ball you throw
always comes back down instead of floating away!
```

TRY REMOVING THE PARSER

If you set `chain = prompt | model` and print the result, you'll get an `AIMessage(content='...', ...)` object full of metadata instead of a plain string. The parser's only job is to reach in and hand you the text. Small, but you'll use it constantly.

1.5 — The same chain, locally (Ollama) — the payoff

Here is the moment LangChain earns its keep. To run the *exact same chain* on a local model, you change **one line**: the model. Everything else — prompt, parser, pipe, invoke — is untouched.

Create `chain_ollama.py`:

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} to a 10-year-old in 2 short sentences."
)
model = ChatOllama(model="llama3.2", temperature=0.3)  # ← the only real change
parser = StrOutputParser()

chain = prompt | model | parser  # identical line

print(chain.invoke({"topic": "gravity"}))  # identical line
```

Differences from the Gemini version: no `load_dotenv()` (local needs no key), and `ChatOllama` instead of `ChatGoogleGenerativeAI`. The chain definition and the call are **character-for-character the same**.

```
python3 chain_ollama.py
```

```
Gravity is like an invisible hand that pulls everything down toward the ground.
That's why when you jump up, you always come back down instead of flying away!
```

THIS IS THE CORE SUPERPOWER

Because both models are `Runnable`s with the same interface, they are interchangeable. The rest of the course leans on this constantly: write a chain once, run it on cloud or local, switch with a one-line edit.

1.6 — A shortcut: `init_chat_model`

Writing out `ChatGoogleGenerativeAI(...)` vs `ChatOllama(...)` works, but LangChain 1.x offers a tidy helper that picks the right class from a `"provider:model"` string. Handy when you want the backend to be configurable:

```
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model

load_dotenv()

# Pick ONE of these – same variable, same downstream code:
model = init_chat_model("google_genai:gemin-2.5-flash", temperature=0.3)
# model = init_chat_model("ollama:llama3.2", temperature=0.3)

print(model.invoke("Say hi in 3 words.").content)
```

WHEN TO USE WHICH

Use the explicit classes (`ChatGoogleGenerativeAI` , `ChatOllama`) while learning — they make the provider obvious. Reach for `init_chat_model` when you want to choose the backend from a config value or environment variable without `if/else` blocks. Both return a normal model Runnable; chains don't care which you used.

1.7 — Common errors & troubleshooting

Symptom	Likely cause & fix
<code>KeyError: 'topic'</code> or <code>Input to ChatPromptTemplate is missing variable 'topic'</code>	You invoked with the wrong key. The dict key must match the <code>{placeholder}</code> exactly: <code>{"topic": ...}</code> for <code>{topic}</code> .
You passed a plain string and got an error	A chain that starts with a <i>template</i> expects a dict , not a string: use <code>chain.invoke({"topic": "gravity"})</code> , not <code>chain.invoke("gravity")</code> .
Output is <code>content='...'</code> <code>additional_kwargs={}</code> instead of clean text	You forgot the parser, or printed the message object. Add <code> StrOutputParser()</code> to the chain, or print <code>result.content</code> .
<code>TypeError: unsupported operand type(s) for </code>	One side of a <code> </code> isn't a Runnable. Wrap plain functions in <code>RunnableLambda(...)</code> , and make sure you're piping the objects (<code>prompt model</code>), not their results.
Gemini key / Ollama connection errors	Same fixes as Phase 0 §0.10: check <code>load_dotenv()</code> + <code>.env</code> for Gemini; make sure the Ollama app/server is running and the model was pulled for Ollama.

1.8 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 2)

Task: Build a **two-input** "translator" chain. The prompt should take *both* a **sentence** and a **language** and ask the model to translate. Run it once on Gemini and once on Ollama, and use `.batch()` to translate three sentences at once.

Target call:

```
chain.invoke({"sentence": "Good morning", "language": "French"})
chain.batch([
    {"sentence": "Hello", "language": "Spanish"},
    {"sentence": "Thank you", "language": "Japanese"},
    {"sentence": "Goodbye", "language": "German"},
])
```

Why: it proves you understand templates with multiple variables, the **prompt | model | parser** pattern, model-swapping, and that `.batch()` works on a whole chain — all the load-bearing ideas of this phase.

Hints (not the full answer):

- A template can hold two blanks: `"Translate '{sentence}' into {language}."`
- The keys in each dict you pass must match the blank names exactly.
- The chain is still just **prompt | model | parser** — nothing new there.
- To swap backends, change only the `model =` line, exactly like §1.5.
- `.batch([...])` returns a *list* of strings, one per input dict — loop over it to print.

Terms introduced this phase

Term	One-line meaning
Framework	A reusable code skeleton you build inside; supplies structure + plumbing.
Runnable	LangChain's universal "plug shape": any object supporting <code>invoke/stream/batch</code> .
<code>RunnableLambda</code>	Wraps a plain Python function so it becomes a Runnable.
LCEL	LangChain Expression Language — composing Runnables with the <code> </code> pipe.
<code> </code> (pipe)	"Run left, feed its output into right." Produces a new combined Runnable.
Chain	Several Runnables piped together; itself a Runnable.
PromptTemplate	A reusable prompt with <code>{blanks}</code> filled from an input dict (Phase 3).
StrOutputParser	A Runnable that extracts the plain text string from a model's reply.
<code>init_chat_model</code>	Helper that builds a model from a <code>"provider:model"</code> string.

AIMessage	The object a chat model returns; its text lives in <code>.content</code> .
-----------	--

PHASE 1 COMPLETE

You now understand the single idea the whole framework rests on: everything is a **Runnable**, and LCEL's `|` snaps Runnables into chains that are themselves Runnables. You built `prompt | model | parser` and ran it on both Gemini and Ollama with a one-line swap. **Next — Phase 2: Working with Models**, where we go deep on chat models vs LLMs, the `invoke` / `stream` / `batch` trio in practice, message roles, and the parameters (temperature, max tokens, and friends) that shape what a model says.